Step 4 — Graceful Degradation When One MCP Server Is Down — Hints

> Back to the step · Hint 1 · Hint 2 · Solution

- Hint 1 — The idea, and where to catch
- Hint 2 — The plan, and proving it

Hint 1 — The idea, and where to catch

A dead server must degrade the team, not crash it. The failure is caught **inside the Web Agent's own node** (really inside `run_specialist`), which returns `("", True)` — empty findings, `unavailable=True`. If you built Step 1's `run_specialist` with the `try/except` + `asyncio.wait_for` around the whole connect-and-loop, you're mostly done: the flag threads through `web_node -> state["web_unavailable"] -> decide_next (skip web) -> run_writer (say so)`.

Hint 2 — The plan, and proving it

```
run_specialist:  wrap connect + initialize + loop in try/except;
                 asyncio.wait_for(session.initialize(), timeout);
                 on failure: return ("", True)   # caught inside the node
web_node:        writes web_unavailable into state
decide_next:     web_unavailable -> route to writer (skip dead server)
run_writer:      web_unavailable -> "(the web status page could not be reached)"
```

Prove it: give `build_graph` an optional `web_args` and point it at a script that doesn't exist. The run finishes from the notes alone, `web_unavailable` is `True`, and the report names the gap. Catching **inside** the node (not in the Supervisor or the graph) is what keeps one dead server from touching the rest of the team. The whole file set is in the Solution.